



SoMa: Identifying, Exploring, and Understanding the DRAM Communication Scheduling Space for DNN Accelerators

Jingwei Cai[†], Xuan Wang^{‡§}, Mingyu Gao^{†¶}, Sen Peng^{‡§}, Zijian Zhu[†], Yuchen Wei[†], Zuotong Wu^{‡§}, and Kaisheng Ma^{†*}
 Tsinghua University[†], Xi'an Jiaotong University[‡], IIISCT[§], Shanghai AI Laboratory[¶] Shanghai Qi Zhi Institute[◆]
 Corresponding Author*

{caijw21,gaomy,zhuzj23,weiy22,kaisheng}@tsinghua.edu.cn, {xuanwang,3123353083}@stu.xjtu.edu.cn

Abstract—Modern Deep Neural Network (DNN) accelerators are equipped with increasingly larger on-chip buffers to provide more opportunities to alleviate the increasingly severe DRAM bandwidth pressure. However, most existing research on buffer utilization still primarily focuses on single-layer dataflow scheduling optimization. As buffers grow large enough to accommodate most single-layer weights in most networks, the impact of single-layer dataflow optimization on DRAM communication diminishes significantly. Therefore, developing new paradigms that fuse multiple layers to fully leverage the increasingly abundant on-chip buffer resources to reduce DRAM accesses has become particularly important, yet remains an open challenge.

To address this challenge, we first identify the optimization opportunities in DRAM communication scheduling by analyzing the drawbacks of existing works on the layer fusion paradigm and recognizing the vast optimization potential in scheduling the timing of data prefetching from and storing to DRAM. To fully exploit these optimization opportunities, we develop a Tensor-centric Notation and its corresponding parsing method to represent different DRAM communication scheduling schemes and depict the overall space of DRAM communication scheduling. Then, to thoroughly and efficiently explore the space of DRAM communication scheduling for diverse accelerators and workloads, we develop an end-to-end scheduling framework, SoMa, which has already been developed into a compiler for our commercial accelerator product. Compared with the state-of-the-art (SOTA) Cocco framework, SoMa achieves, on average, a $2.11\times$ performance improvement and a 37.3% reduction in energy cost simultaneously. Then, we leverage SoMa to study optimizations for LLM, perform design space exploration (DSE), and analyze the DRAM communication scheduling space through a practical example, yielding some interesting insights. Moreover, SoMa has been open-sourced at <https://github.com/SET-Scheduling-Project/SoMa-HPCA2025>.

I. INTRODUCTION

In order to process a variety of tasks with better performance and accuracy, DNNs are rapidly becoming more complex and heavy [13], [14], [19], [41], [51]. Accelerators [23], [25], [26], [31], [34], [38] with more computing units, larger buffers, and higher memory bandwidth have been developed to accelerate these DNN workloads.

However, under modern semiconductor process, the rate of increase in DRAM bandwidth lags behind the rate of transistor density growth, which is a long-standing issue [46], [54]. This disparity is even more pronounced in DNN accelerators [46],

[60], which are more specialized and have a larger proportion of dedicated computing elements than traditional chips like CPU. Thus, DRAM communication is increasingly becoming a performance bottleneck in DNN computation [46], [60]. To mitigate this bottleneck, accelerators are equipped with increasingly larger on-chip buffers [18], [23], [25], [31], [34], [38], which provide opportunities to optimize DRAM communication by exploiting reuse opportunities in DNNs.

Several works leverage buffer resources to reduce DRAM accesses under the paradigm of “layer fusion” [3], [7], [8], [29], [33], [37], [49], [60]. This approach involves buffering the feature maps (fmaps) produced by earlier layers on-chip, allowing subsequent consuming layers to read them directly. This strategy avoids the overhead of first writing back to DRAM and then reading from it, thereby reducing DRAM access costs. This optimization paradigm holds immense potential; for instance, Cocco [49] achieved performance improvements ranging from 1.89% to 50.33% by merely exploring the option of which layers to fuse. Beyond this, there are numerous dimensions worth exploring under this paradigm, such as execution order and execution granularity. However, like Cocco, most existing studies [3], [37] have only focused on a small portion of this optimization space. Therefore, we believe that the complex optimization dimensions within the layer-fusion paradigm have yet to be clearly delineated or defined, much less thoroughly explored and understood in the context of the entire layer-fusion optimization space.

While reducing DRAM access is an important approach for optimizing DRAM communication, we identify another optimization approach that has been largely overlooked in the field of DNN scheduling: prefetching and delayed storing, i.e., adjusting the timing of fetching/storing data from/to DRAM to be sometime earlier/later. We focus on this approach and believe it has potential based on an insightful observation: in modern DNN networks, the ratio of DRAM bandwidth demand to computing demand varies significantly across different layers (see Fig. 3). After layer fusion, the overall ratio of DRAM bandwidth demand to computing demand for different tiles (computing unit) varies even more (see Fig. 3). This observation indicates that with the application of layer fusion, DRAM bandwidth usage during the entire computing

process becomes very uneven—sometimes leading to congestion due to high demand and sometimes causing bandwidth resource waste due to low demand. This motivates us to apply prefetching-and-delayed-storing techniques to alleviate the uneven DRAM communication load. However, choosing the appropriate timing for prefetching and storing is a non-trivial problem. Clearly defining, thoroughly exploring, and understanding this paradigm is a significant challenge.

“Layer Fusion” and “Prefetching and Delayed Storing” each have their own optimization spaces, but they are not independent and are intricately coupled. This is reflected in the following points: 1) both paradigms trade buffer usage for DRAM communication optimization, leading to competition for buffer usage, and 2) layer fusion affects the types and quantities of data that need to communicate with (i.e., prefetch from and store to) DRAM. Therefore, we define the complex space formed by these two paradigms as the DRAM Communication Scheduling Space.

After identifying the challenges and optimization potential of DRAM communication scheduling, we make the following contributions to identify, explore, and understand DRAM Communication Scheduling Space.

We first introduce a Tensor-centric Notation with two categories and a total of six attributes to encode the scheduling schemes in the DRAM Communication Scheduling Space; then we show how to parse each encoded scheme into actual hardware behaviors. Based on this notation, we define and illustrate the DRAM Communication Scheduling Space and the complex trade-offs behind it. Existing works can be described using our notation, representing only a small subset within the space we have defined. To the best of our knowledge, this work is *the first* to comprehensively define and analyze the DRAM Communication Scheduling Space.

To thoroughly and structurally explore the DRAM Communication Scheduling Space defined by the Tensor-centric notation, we develop an end-to-end framework, SoMa, which employs a Buffer Allocator, a two-stage simulated annealing (SA) exploration engine, and an accurate simulator to conduct a structured and efficient exploration of the space. We have successfully built a complete compilation flow based on SoMa, from model input to instruction generation, for an accelerator approaching mass production.

Then, we conduct extensive experiments on different workloads (including CNNs and LLMs), hardware configurations, and batch sizes, demonstrating an average of a $2.11\times$ performance improvement and a 37.3% reduction in energy costs compared to the SOTA framework, Cocco. We analyzed the experimental performance of LLMs and uncovered several intriguing phenomena: (1): For the decode stage, the optimization potential of DRAM scheduling is minimal due to its extremely low compute density, which imposes a pure DRAM bandwidth demand on the accelerator. (2): For the decode stage, increasing the batch size does not consistently improve computational utilization. As the batch size grows, the increasing size of the KV cache becomes comparable to that of the weights, diminishing the benefits of further increases in

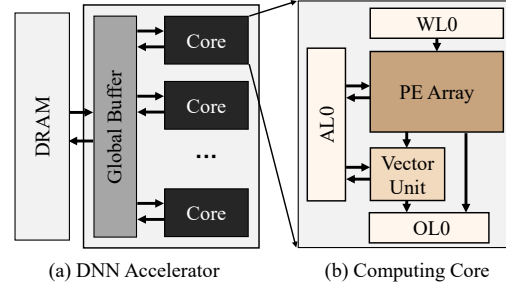


Fig. 1. DNN Accelerator Template

batch size for improving compute density. In addition, we use SoMa to explore and analyze the architectural design space, gaining some interesting insights. For example, with small batch sizes, DRAM bandwidth plays an irreplaceable role. However, with SoMa, the importance of the buffer becomes increasingly prominent as the batch size increases. Moreover, we present a practical execution graph comparison between Cocco and SoMa to enhance understanding of the trade-offs underlying the DRAM Communication Scheduling Space.

II. HARDWARE BASIS

In this section, we introduce a generic large-scale DNN accelerator template (Fig. 1) and the corresponding abstract instruction system adopted in this paper, which represents many mainstream commercial DNN accelerators [6], [16], [24]–[26], [34], [38].

As shown in Fig. 1(a), the template primarily consists of DRAM, a Global Buffer (GBUF), and several cores. The GBUF is shared among all cores. As shown in Fig. 1(b), each core has private small buffers/register files ($WL0$, $AL0$, $OL0$) for rapid access by computing units. The PE Array is used for computing GEMM/Conv operations, and the Vector Unit is designed for computing other vector/scalar operations, such as element-wise addition, pooling, layer normalization, etc.

Although specific instructions vary significantly among these accelerators, they still share apparent common patterns. Based on these common patterns, we abstract three instructions: load, store, and compute. The “load” and “store” instructions refer to moving data from DRAM to the GBUF and from the GBUF to DRAM, respectively. The “compute” instruction refers to the operations performed on a tensor/vector. In accelerators, a tensor is often divided into smaller tensors, which are sequentially processed by the core group. Each small tensor is further split into smaller sub-tensors for parallel processing by the cores within the core group. The specific operations involved include loading ifmaps and weights from GBUF into the local buffer of each core, performing the computations, and then writing the computed ofmaps back to the GBUF. Since these instructions typically occur in sets and are synchronized, and as this study focuses on optimizing DRAM communication, we abstract them into a single “compute” instruction. In our DRAM-COMPUTE diagram (e.g., Fig. 4 Bottom), “load & store” instructions and “compute” instructions can be respectively represented by

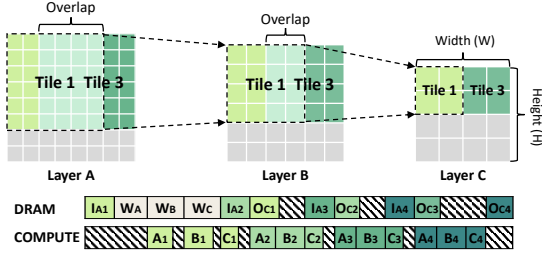


Fig. 2. A Practical Layer-fusion Group (LG) Example

tensor blocks in the DRAM row and the COMPUTE row. The start and end of any instruction can serve as markers for the beginning of another instruction (Fig. 4 Right).

III. IDENTIFY DRAM COMMUNICATION OPTIMIZATION OPPORTUNITIES

The GBUF plays a crucial role in optimizing DRAM communication within DNN accelerators, and research on how to use it to optimize DRAM communication has been a hot topic since the advent of DNN accelerators [10]–[12], [26], [35]. However, most current studies focus on optimizing dataflow for small-scale DNN accelerators, specifically on how to tile a single layer into small parts and adjust their computing order to optimize DRAM communication [20], [21], [27], [32], [36], [42], [48], [52], [55], [57].

As DNN accelerators have evolved, architects have equipped them with increasingly larger GBUFs, with capacities reaching tens or even hundreds of megabytes [2], [16], [24], [25], [31], [34]. Such large buffers can accommodate most individual layers of most networks, significantly reducing the effects of optimizing single-layer dataflow for DRAM communication. Therefore, developing new techniques to fully leverage the rapidly growing on-chip buffer resources to optimize the increasingly bottlenecked DRAM bandwidth is crucial and offers significant opportunities. Next, we will analyze the challenges and optimization opportunities in DRAM communication scheduling.

A. Layer Fusion

Layer fusion is an important paradigm for using buffers to optimize DRAM communication [3], [37], [49], [59]. Fig. 2 illustrates the computation and DRAM access behavior of a simple three-layer network under layer fusion, where multiple fused layers are computed in a fine-grained manner sequentially, relying on on-chip buffers for switching fmaps. The optimization dimensions within this paradigm are numerous and complex, but have not been clearly depicted and analyzed. A straightforward optimization dimension is which layers to fuse, which affects both DRAM access savings and buffer occupancy. This is the primary focus of most existing studies [49], [59]. Another evident optimization dimension is the computing granularity of each fused layer group, i.e., the size of each computing tile (Fig. 2). This affects buffer occupancy, halo overlap overhead, and the effectiveness of intra-core optimizations (detailed analysis is in Sec. IV-A1).

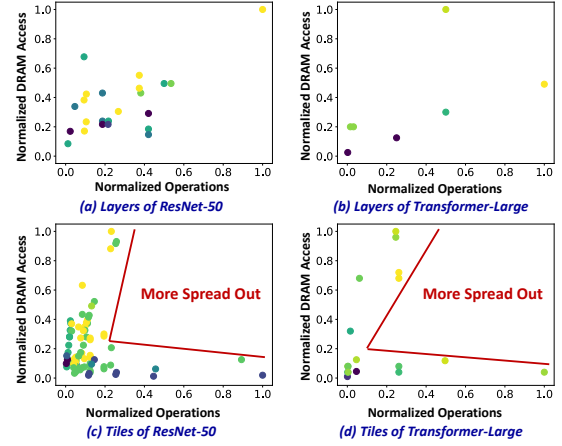


Fig. 3. (a) and (b) show the normalized DRAM access and the normalized operation number for each layer in ResNet-50 and Transformer-Large, respectively (each point represents a layer). (c) and (d) show the normalized DRAM access and the normalized operation number for each smallest computing unit (Tile) of ResNet-50 and Transformer-Large, respectively, scheduled using the SOTA Cocco Framework (each point represents a Tile). **The darker the color, the more identical overlapped points there are.** The normalization method involves dividing the value of each point by the maximum value among all points (DRAM access and operations are independently normalized). We use the default edge accelerator and batch size 1, as introduced in Sec. VI-A

DeFENIS [37] has analyzed this dimension but has not jointly explored the above two dimensions. Most other studies address this dimension using heuristic rules [49], [59], overlooking optimization opportunities within this space. We can see that even these two dimensions lack systematic joint exploration, let alone additional dimensions (introduced in Sec. IV-A1) that have been overlooked by existing works.

B. Prefetching and Delayed Storing

Besides layer fusion, we have identified another optimization opportunity overlooked by existing literature: prefetching and delayed storing. We argue this paradigm also has great potential based on the following insightful observation.

As shown in Fig. 3(a) and (b), in modern DNNs, the DRAM access demand and computing requirements vary significantly across different layers. Unfortunately, after layer fusion, the variance in the memory access to computation ratio for each tile becomes even more pronounced (see Fig. 3(c) and (d)), which are more spread out along the axes compared to their counterpart in Fig. 3(a) and (b). This indicates a larger number of DRAM-access-intensive and compute-intensive tiles, with fewer tiles having balanced demands. Specifically, this is because, within a fused layer group, the first tile of each layer with weights needs to load weights (e.g., A_1, B_1, C_1 in Fig. 2), which often results in high DRAM demand, corresponding to the points near the Y-axis in Fig. 3(c) and (d). The subsequent tiles do not need to load weights (e.g., A_2, B_2, C_2 in Fig. 2). Additionally, many fmaps' DRAM access requirements are eliminated as a result of fusion (e.g., the ofmaps of A_i and B_i). Consequently, many tiles even have no DRAM access demand (e.g., B_2, B_3, B_4), corresponding to the points near the X-axis in Fig. 3(c) and (d).

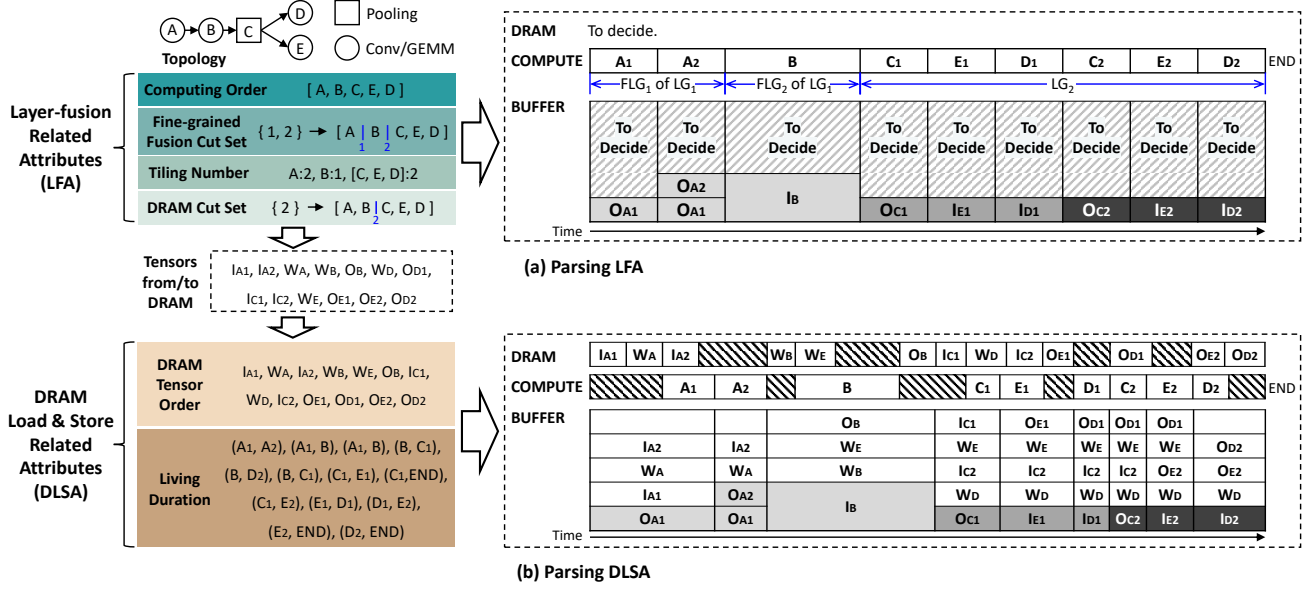


Fig. 4. Parsing an Example Encode of a Five-Layer Network into Actual Scheduling Schemes. The DRAM, COMPUTE, and BUFFER in the right part show the DRAM access, workload computation, and buffer usage, respectively. We use M_i to represent the i th tile of layer M , and $I/O_{M,i}$ to represent the ifmaps and ofmaps of M_i . W_M represent the weights of layer M . In the BUFFER, blocks with the same background color represent the same data.

This severe imbalance between DRAM access and compute demands makes the overlap between DRAM access and compute under the traditional double-buffer strategy (prefetching data in the previous tile and storing data in the next tile) insufficient, especially in the context of layer fusion. For example, Fig. 2 shows the DRAM communication under the traditional double-buffer strategy. As seen, there is a waste of DRAM bandwidth, and the computing resources are severely stalled. To better demonstrate the severity and prevalence of this challenge, we analyze the workloads, batch sizes, and platforms in Sec. VI-A. With the SOTA Cocco Scheduling Strategy, the DRAM and computation utilization rates for the case in Fig. 2(c) are 52.69% and 62.64%, respectively, while in Fig. 2(d) they are 72.45% and 45.84%. These utilization are defined as the ratio of the sum of all DRAM tensors/computing tiles time to the total runtime. This result indicates that neither resource is fully utilized, leaving significant opportunities for overlap.

Based on the above observation and analysis, we find that controlling the timing of data prefetching and storing to utilize idle DRAM bandwidth and alleviate peak-time pressure has great potential. For example, suppose that there are two consecutive identical Layer-fusion Groups (LG and LG') (Fig. 2 shows LG , and LG' is not shown). By prefetching LG' 's W_A , W_B , and W_C during the DRAM idle time corresponding to B_3 , B_4 , and C_4 in LG , the computing stall at the start of LG' can be resolved. Additionally, starting to load I_{A3} and I_{A4} in LG earlier can also erase the stalls before A_3 and A_4 in LG . However, achieving precise and automatic control over prefetching and delayed storing is non-trivial. Clearly defining, thoroughly exploring, and understanding this paradigm is a significant challenge.

C. Combine Them Together

“Layer Fusion” and “Prefetching and Delayed Storing” are two optimization paradigms with complex interrelationships, which lie in two aspects. The first aspect lies in the fact that both optimization paradigms inherently trade buffer usage for DRAM communication optimization, leading to competition for buffer resources between the two. Additionally, the layer fusion choice affects DRAM communication requirements, which in turn impacts the optimization space of prefetching and delayed storing. Therefore, these two paradigms exhibit complex interactions within the DRAM Communication Scheduling Space. Efficiently and structurally exploring this space presents a major challenge. Addressing this challenge and gaining a deep understanding of these interactions form the key objectives of this paper.

IV. TENSOR-CENTRIC NOTATION

A. Encoding Format and Parsing Methods

In this section, through a practical example of a five-layer network (A to E) shown in Fig. 4, we explain how the proposed Tensor-centric Notation translates layers into fine-grained tensor computations, DRAM accesses, and buffer usage, as well as the trade-offs associated with different encoding choices.

As shown in Fig. 4, the notation has six attributes, which can be divided into two categories: Layer-Fusion-related Attributes (LFA) and DRAM-Load-and-Store-related Attributes (DLSA). Consequently, the overall parsing process is also divided into two stages. The first stage involves parsing the LFA to determine: 1) the computing granularity and order, as well as the buffer occupancy of tensors reused on-chip, and 2) all tensors requiring DRAM interaction. The second

stage involves parsing the DLSA to derive the specific timing and buffer occupancy details for all tensors that must be loaded/stored from/to DRAM.

1) *Layer-Fusion-related Attributes*: LFA includes **Computing Order**, **Fine-grained Layer-fusion Cut (FLC)**, **Tiling Number**, and **DRAM Cut**, which we will discuss in sequence.

The first attribute, Computing Order, arranges these layers into a serial sequence, representing their coarse-grained execution order. Thus, a valid Computing Order cannot have any dependency that goes from right to left, as this would result in a scenario where the data needed by a layer calculated earlier is not yet computed. We can see an example of the Computing Order in Fig. 4; swapping the order of D and E remains valid, but swapping A and B does not.

The second attribute, Fine-grained Layer-fusion Cut (FLC), records the cut locations, which cut layer sequences into Fine-grained Layer-fusion Groups (FLGs). All FLCs together constitute the FLC Set ($\{1, 2\}$ in Fig. 4). Each FLG possesses an attribute, Tiling Number (the third attribute), which determines the computing granularity of the FLG. Layers within an FLG are processed continuously at the granularity of the tile rather than completely finishing the first layer before starting the second, thereby saving on-chip buffer overhead. For the example in the left top of Fig. 4, the layers are cut into three FLGs ($[A]$, $[B]$, $[C, E, D]$) with Tiling Numbers 2, 1, 2, respectively. Within the FLG $[C, D, E]$, each layer is divided into two tiles, which are then processed sequentially. Given the Tiling Number, we use a heuristic strategy to partition each layer into computing tiles along the multiple dimensions. Specifically, it prioritizes tiling the batch dimension since it does not produce halo overlap costs, followed by the height and width of the ofmaps, keeping them as equal as possible to reduce overlap. The reason for not splitting the channel dimension is that splitting the channel would prevent the next layer from accessing all channels, making it impossible to fuse more than two layers. For example, in Fig. 2, with a batch size of 1 and a Tiling Number of 4, we split the height and width dimensions each by 2 (the channel dimension is not shown). It is worth emphasizing that for intermediate layers involving operations like convolutions or poolings, which produce halo overlaps, the size of each tile may be larger than $1/\text{Tile}$ of the size of the fmaps (e.g., *Layer A* and *B* in Fig. 2). The specific method for determining each tile's size of intermediate layers considering halo overlap influence has already been proposed in Cocco [49] and DeFENIS [37], so we directly adopt their methods. The trade-off associated with the Tiling Number primarily concerns the balance between buffer usage, halo overlap overhead, and computing efficiency within the core array. The finer-grained tiles imply reduced buffer usage, but if they involve layers that produce overlaps, they may incur more extra computation and memory accesses. Additionally, the computing efficiency within the core array may decrease, as smaller tiles imply reduced on-chip reuse opportunities [57].

By parsing the above three attributes, we can derive the entire computation sequence (the COMPUTE row in Fig. 4).

We define certain specific FLCs as DRAM Cuts, which

cut layer sequences Layer-fusion Groups (LGs). Dependencies between different LGs require data to be sent to DRAM and then loaded back for computation. All DRAM Cuts together constitute the DRAM Cut Set ($\{2\}$ in Fig. 4). For example, C depends on B , and there is a DRAM cut between them. Therefore, C can only load ifmaps from DRAM after B has sent its ofmaps to DRAM. Thus, all the requests for each tile's interaction with DRAM can be determined as follows: if a layer has weights, it has a weight-load request. For example, all layers except C (pooling layer has no weights) require their weights to be loaded from DRAM (represented as W_M in the left part of Fig. 4). If a layer has forward dependencies that span different LGs (or its input is the overall network input), then the ifmaps of its related tiles need to be loaded from DRAM (I_{Mi} in the left part of Fig. 4). If it has backward dependencies that span different LGs (or its output is the overall network output), then the ofmaps of its related tiles need to be written back to DRAM (O_{Mi} in the left part of Fig. 4). The trade-off involved in DRAM Cuts primarily concerns the balance between buffer requirements and the volume of DRAM access. Generally speaking, the more fused layers there are (the fewer the DRAM Cuts), the lower the number of DRAM accesses will be, but the demand for buffer capacity will increase.

The remaining fmaps corresponding to the dependencies that do not cross DRAM cut can be directly reused on-chip. The buffer occupancy duration for such data ranges from the production of the ofmaps tile to the consumption of the tile. For the example in Fig. 4(a), there is an FLC between A and B , thus the ofmaps of A are stored on-chip. Fig. 4(a) only shows the buffer allocation for on-chip data transfers, while all tensors related to DRAM communication are not displayed (to determine in the DRAM Load & Store Phase).

Since the DRAM Cut Set is a subset of the FLC Set, each LG contains one or more FLGs. When data dependencies exist between different FLGs, the producing layers in the former FLG need to aggregate the ofmaps from different tiles of the layers before they can be used by the consuming layers in the latter FLG. Once the computation for an FLG is completed, the weights of the layers in the FLG can be released, freeing up some buffer space. For the example in Fig. 4, the weights of A can be released after all its tiles are computed. B must wait until A to finish computing all its ofmaps before it can start its computation. The above analysis demonstrates that the FLC primarily involves a trade-off in buffer occupation. Specifically, the FLC can free up some buffer space occupied by weights at the cost of fmaps accumulation.

2) *DRAM-Load-and-Store-related Attributes*: DLSA includes **DRAM Tensor Order** and **Living Duration**, which will be introduced in serial.

The DRAM Tensor Order attribute determines the access sequence for all DRAM tensors, such as the tensors in dashed squares which is in the middle of Fig. 4 Left.

Every DRAM tensor has an adjustable Living Duration attribute, which is a 2-element tuple (*Start*, *End*). *Start* and *End* are tile IDs. This attribute has dual implications: first,

within this time frame, the required buffer is allocated to this tensor. For the example in Fig. 4, the Living Duration of W_E is (B, D_2) . Therefore, W_E lives in buffer from Tile B to D_2 . Second, it schedules the timing for loading and storing. Specifically, for ifmaps and weights, the *End* is fixed at the next tile of the last tile that requires them, indicating when they can be released. The *Start* indicates when this tensor can start being loaded from DRAM. For ofmaps, the *Start* is fixed at the tile that produces them, indicating that once this tile finishes, storing to DRAM can begin. The *End* signifies the tile by which the transfer must be completed; otherwise, the tile should stall. For example, the *End* of O_{E1} is D_1 , so if its store is not completed, D_1 cannot start. Although *Start* indicates when loading or storing can begin, the actual start time depends on whether the required data is ready and whether the loads and stores preceding this tensor in the DRAM Tensor Order have been completed. For example, the *Start* of I_{C2} is C_1 , but the loading of the preceding W_D takes a long time, causing the actual load of I_{C2} to begin only in the middle of E_1 .

B. Space Size Comparison

As introduced above, each scheme is encoded by six variable attributes. Thus, our Tensor-centric Notation constructs a six-dimensional vast optimization space. In contrast, if we map the representable scheduling schemes of the SOTA Cocco [49] into our notation, only Computing Order and DRAM Cut can change, with the FLC Set being identical to the DRAM Cut Set. The other four attributes are determined by heuristic strategies, either explicitly or implicitly, making its explorable optimization space much smaller than ours. Moreover, DeFENIS [37] proposes a simulator and demonstrates the performance and energy efficiency of: 1) different LGs with heuristically determined Tile Numbers, and 2) the same LGs with Tile Numbers enumerated at certain intervals. They analyze some characteristics of these two attributes individually. However, they do not jointly explore the space composed of these two attributes, let alone the other four dimensions. As a result, the schemes it touches are far fewer than ours.

V. SoMa FRAMEWORK

A. SoMa Overview

As shown in Fig. 5, SoMa is an end-to-end DNN scheduling framework. SoMa takes as inputs: 1) hardware configuration, such as the number and organization of cores, DRAM bandwidth, buffer size, etc.; 2) framework configurations, such as the optimization goal and searching hyperparameters, etc.; 3) a DNN model description file generated by high-level frameworks like PyTorch. After scheduling, SoMa outputs: 1) instructions; 2) reports on energy costs and latency; 3) a detailed scheduling scheme.

The optimization objective of SoMa is $Energy^n \times Delay^m$, where n and m are adjustable parameters that prioritize energy efficiency or performance according to different needs. *Energy* and *Delay* refer to the total energy costs and latency for processing a batch of samples, respectively. Thus, small

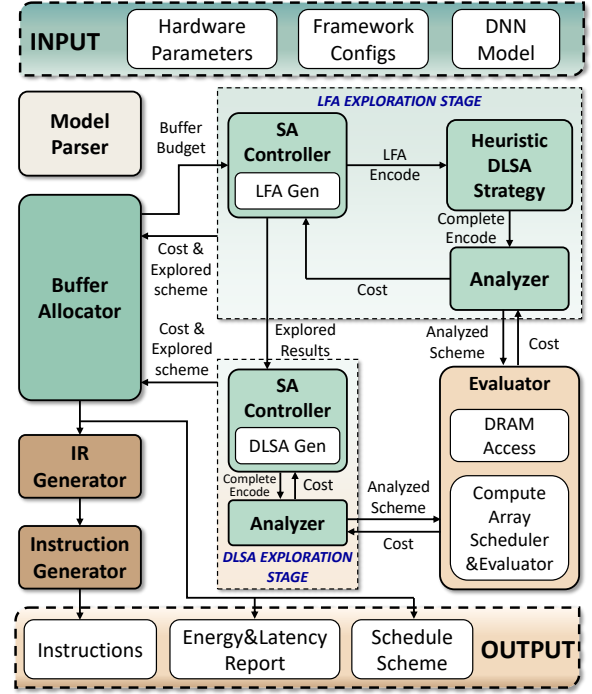


Fig. 5. SoMa Framework

batch sizes can evaluate SoMa's effectiveness in latency-centric scenarios, while larger batch sizes can be used for throughput-centric scenarios.

Fig. 5 shows that the key exploration process is governed by three main components: the Buffer Allocator, the LFA Exploration Stage, and the DLSA Exploration Stage. The Buffer Allocator controls the outermost iteration, each involving a complete two-stage top-down exploration. Based on the respective effects and buffer usage of the two stages in the previous iteration, the Buffer Allocator adjusts the buffer budget for the two stages competing for buffer usage in the current iteration. The two-stage top-down exploration involves separately varying and searching LFA and DLSA in each stage, with each stage using SA for independent searches.

The reason for using a two-stage search with a Buffer Allocator is that: 1) as analyzed in Sec. III-C, a single change in LFA can significantly impact DRAM tensors (e.g., adding or removing a DRAM Cut or changing the Tiling Number), thus previously optimized DLSA attributes may become sub-optimal or even invalid. This makes it difficult to retain good solutions obtained by varying DLSA attributes while continuously varying LFA. 2) The two-stage exploration with a Buffer Allocator can produce strong synergistic effects. DRAM access has a significant impact on performance and energy costs. Therefore, we observe that the first stage's optimization tends to minimize fmaps-related DRAM access, which reduces the optimization difficulty and increases the potential for the second stage optimization. This is mainly because a) the large proportion of weights is beneficial for optimization, as weights have fewer dependency constraints

and can be adjusted more freely, while fmaps have more complex dependencies with a narrower range of adjustments; b) more extensive layer fusion and reduced DRAM access result in purer compute time, providing greater optimization opportunities for the second stage. Therefore, we found that dividing the optimization into two stages can produce good synergistic effects. The only risk is that the first stage might occupy too much buffer space, excessively limiting the optimization space for the second stage. We address this potential risk through the Buffer Allocator.

The optimal results can be directly output from the Buffer Allocator as scheduling schemes and reports, or they can be converted into a more easily parsable intermediate representation (IR) through our IR Generator module. This IR can then be fed into the Instruction Generator module to generate actual instructions.

B. Buffer Allocator

In the first iteration, we conduct a complete two-stage search, with the only constraint being that the buffer usage does not exceed the hardware buffer capacity. We record the maximum buffer usage of the scheme explored in the first stage ($Buffer_{max}$), as well as the best overall encoding scheme and its cost ($Cost_{best}$). In subsequent iterations, the buffer usage limit for the first stage is reduced by $a\%$ (10% used in the following experiments) of $Buffer_{max}$ each time (solutions exceeding this limit are deemed invalid), and the overall cost ($Cost_{temp}$) is recorded. If $Cost_{temp}$ is better than $Cost_{best}$, $Cost_{best}$ and the optimal encoding scheme are updated accordingly. The iteration stops when the costs of the optimal solutions found in two consecutive iterations do not exceed $Cost_{best}$. The rationale behind using this iteration to allocate buffers between the two stages for overall optimization is that while the performance of both stages improves with increased buffer size, the rate of improvement slows as buffer size grows. Therefore, adjusting the buffer allocation in small increments helps effectively find the sweet spot that maximizes the combined performance of both stages.

C. LFA & DLSA Exploration Stage

Both LFA and DLSA employ SA to explore this space. The key factors in SA are the initial solution, cooling schedule, and operators. The initial solution and operators are discussed in the following sections, while the cooling schedule is described here. Starting from an initial solution, each iteration randomly selects an operation to modify the encoding and evaluates it. If the new scheme's cost (c') is higher than the previous cost (c), it is accepted with probability $p = e^{\frac{c-c'}{T_n}}$, where T_n is the temperature at iteration n . Otherwise, the scheme is always accepted. The temperature at iteration n is given by $T_n = T_0 \frac{1-\alpha}{1+\alpha \frac{n}{N}}$, where T_0 is the initial temperature and α is the cooling rate. The total number of iterations is $N = \beta X$. For the first stage, β and X are set to 100 and the number of layers, respectively. For the second stage, they are set to 1000 and the number of DRAM tensors, respectively. We also support setting an additional termination time. Once this

time is reached, the algorithm performs Y more iterations, accepting only improved solutions.

1) *LFA Exploration Stage*: In this stage, The initial solution consists of each layer forming its own independent LG and FLG (e.g., both FLG and LG are 1, 2, 3, 4 as shown in Fig. 4), meaning no fusion is applied. The tile number is set to the minimum granularity, corresponding to the size required for the core array to perform parallel computation. Then, the SA operators transform the LFA, while the DLSA is determined using a classical double-buffer strategy (as introduced in Sec. III-C). The specific operators are as follows:

Change Computing Order: Randomly select a layer and change its order to another valid location.

Change Tiling Number: Randomly select an FLG and multiply or divide its Tiling Number by 2.

Add/Delete An FLC: Randomly add or delete an element in FLC Set. Specifically, adding an FLC means cutting an FLG into two FLGs with the same Tiling Number attribute as the original FLG. Removing an FLC means merging two FLGs into one, with the new FLG's Tiling Number inherited probabilistically based on the layer count ratio of the original two FLGs.

Add/Delete A DRAM Cut: Randomly add or delete an element in the DRAM Cut Set. The added element must be in the FLC Set.

2) *DLSA Exploration Stage*: In this stage, the initial solution adopts the best scheme explored by the previous stage, with the LFA attribute remaining constant. The SA controller then primarily focuses on searching within the DLSA for the DRAM tensors corresponding to this selected LFA. The specific operators are introduced as follows:

Change DRAM Tensor Order: Randomly select a DRAM tensor and change its order to another valid location.

Change Living Duration: Randomly select a DRAM tensor and randomly change its *Start* (for ifmaps and weights) or *End* (for ofmaps). For example, in Fig. 4(b), by reducing the *Start* of W_B by 1 from B to A_2 , the STALL between A_2 and B can be eliminated, and W_B will be included in the buffer associated with A_2 .

Notably, in each operation, the probability of selecting a DRAM tensor is proportional to its size since larger tensors generally have a greater impact on performance and buffer utilization, warranting more transformation opportunities.

D. Evaluator

In this section, we introduce an accurate evaluator, capable of evaluating various scheduling schemes, described using our Tensor-centric Notation across different hardware configurations in terms of energy cost and latency.

The evaluation process follows a local-to-global approach, first assessing each computing tile and DRAM load/store request (DRAM tensor) individually and then conducting an overall assessment.

For each computing tile (e.g., A_1 in Fig. 4), the ifmaps and weights have been prefetched into the GBUF, and the ofmaps are written back to the GBUF. From a macro perspective, the Core Array Scheduler explores how to further

divide this tile into sub-tiles for computation by each core (as introduced in Sec. II), aiming to maximize parallelism and data reuse. The corresponding Evaluator assesses each interaction between GBUF and L0 buffers, as well as computations, while accounting for dependencies to evaluate overall performance and energy consumption. The corresponding energy costs and computing time of the searched optimal scheme are taken as the tile's energy costs and computing time. As this area of research is well-established [20], [21], [28], [32], [36], [42], [52], [55], [57], we adopt a classic scheduler and evaluator for this purpose [32], [42].

Each DRAM communication tensor's energy costs are calculated by summing the products of read and write data volumes with their respective unit energy costs. The read and write times are calculated by dividing the data volumes by the respective bandwidths.

The total energy cost is calculated by summing up the energy costs of the above sub-components, similar to existing classical works [32], [42], [57]. The total computing time is derived based on the evaluated times of all computing tiles and DRAM tensors using the following method:

For each DRAM tensor, it can start execution only when the following three conditions are met: 1) the preceding DRAM tensor has been completed; 2) for ifmaps or weights, their *Start* must be smaller than or equal to the current tile ID; and 3) for ofmaps, it must wait until its generating computing tile has finished. For example, in Fig. 4, although I_{C2} 's *Start* is C_1 , the preceding DRAM tensor (W_D) is not completed until E_1 , so it can only start at the middle of E_1 . Moreover, W_B has a *Start* of B . Although the previous DRAM tensor (I_{A2}) has already been completed, it must still wait until A_2 finishes before it can begin.

Each computing tile can start execution only when the following conditions are met: 1) all required data (ifmaps, weights, etc.) are ready. For the example in Fig. 2, A_1 , B_1 , and C_1 cannot follow their respective preceding computing tile immediately because the required data are not ready at the end of the preceding computing tile. 2) All DRAM tensors with *End* less than or equal to the tile must be completed. For the instance in Fig. 4, D_1 cannot follow E_1 because O_{E1} 's *End* is D_1 , and D_1 cannot start until O_{E1} has finished execution.

E. Generality and Portability of SoMa

The proposed encoding and SoMa framework exhibit excellent generality for two main reasons: 1) the template depicted in Fig. 1 is highly general, encompassing many accelerators from both the industry [9], [25], [26], [34], [53] and academia [12], [17], [30]; 2) The hardware behavior information encoded by our notation is very general (i.e., computing, data loading, and storing, instruction dependency, etc.).

Our SoMa also possesses excellent portability, due not only to the aforementioned reasons but also because our framework is designed with a robust modular architecture. This design allows for easy adaptation to different accelerators, which may have distinct core micro-architectures, by simply replacing the relevant Core Array Scheduler & Evaluator modules and

Instruction Generation module. We have developed a comprehensive compilation flow for our accelerator [1], which can serve as a concrete example for porting to other accelerators.

F. Open-source of SoMa

We have now open-sourced files and documentation for key stages to illustrate the entire SoMa Compiler workflow at this link [1]. In the future, we plan to set up a small-scale cloud platform. This platform will allow users not only to access the open-sourced SoMa scheduler but also to modify or even replace our scheduler (as long as the output is converted to IR format), enabling it to be translated into instructions that can run on the chip. Given that many existing accelerators (e.g., TPU [26]) do not offer such low-level API access, we believe this platform can significantly advance related research.

VI. EVALUATION

A. Experiment Setup

1) *Hardware Configuration*: To comprehensively evaluate the effects of SoMa, we utilize both edge and cloud hardware platforms: we set the edge computing power to 16 TOPS (referencing Qualcomm Snapdragon 8 Gen 3's 15 TOPS [44] and Apple A15's 15.8 TOPS [4], A16's 17 TOPS [5]), and the cloud to 128 TOPS (referencing NVIDIA Orin's 138 TOPS [39] and TPU V4i's 136 TOPS [25]). Buffer and DRAM bandwidth are set to 8MB and 32MB, and 16GB/s and 128GB/s, respectively, based on our DSE results in Fig. 7. This configuration allows accelerators to achieve outstanding performance with reasonable hardware resources. We also thoroughly tested SoMa's performance under different buffer sizes and DRAM bandwidths (as shown in Fig. 7). The default process technology is TSMC 12nm, with an operating frequency of 1GHz. All unit energy costs for different operations required by the Evaluator (as introduced in Sec. V-D) are obtained from the actual RTL code synthesis and simulation during our accelerator development. The optimization goal is set as $\text{Energy}^1 \times \text{Delay}^1$, as both energy costs and performance are crucial in inference accelerators [16], [22].

2) *Workload*: To comprehensively evaluate SoMa effects and analyze the trade-offs behind different DRAM communication scheduling schemes, we scale batch size from 1 to 64, covering latency-sensitive scenarios to throughput-centric scenarios as introduced in Sec. V-A. In our experiments, ResNet-50 [19], ResNet-101 [19], Inception-ResNet-v1 (IRes) [47], Randwire [56] and GPT-2 [45] are chosen as workloads. ResNet-50 is chosen because it takes classical residual structures widely employed in many DNNs. ResNet-101 is chosen because it has a similar structure to ResNet-50 but with a larger number of layers. The Inception-ResNet-v1 and Randwire are selected to represent the DNNs with wider and more complex structures. GPT-2 is selected to represent language-processing DNNs. Since GPT-2 has various versions for different scenarios, we use GPT-2-Small with a token length of 512 (prefill 512 and decode the 513th) for the edge platform, and GPT-2-XL with a token length of 1024 (prefill 1024 and decode the 1025th) for the cloud platform.

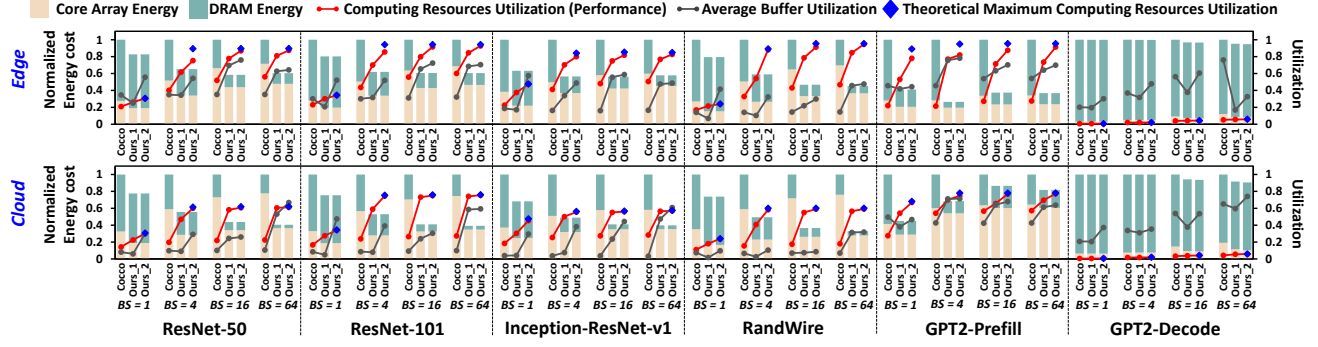


Fig. 6. Overall Comparisons between Cocco and SoMa (Ours). Ours_1 and Ours_2 demonstrate how the efficient solution is progressively optimized through the first and second stages. **Computing Resources Utilization** equals to $Util(\text{Actual Evaluated Latency})$, where $Util(t) = \text{Total Number of Operations in Network} / (\text{Peak Hardware Computing Power} \times t)$. Thus, the utilization can be viewed as a measure of performance. **Average Buffer Usage** is calculated as $\sum_{Tile} (\text{Buffer Usage} \times \text{Computing_Time}_{Tile}) / \text{Total Comp Time}$. The **Theoretical Maximum Computing Resources Utilization** (represented by **Blue Diamonds**) is used to measure the theoretical maximum optimization possible in the second stage. Specifically, it is $Util(\text{MIN}\{\sum_{Tile} \text{Computing_Time}_{Tile}, \sum_{Tensor} \text{DRAM_Access_Time}_{Tensor}\})$, without considering dependencies (e.g., $Util(\text{MIN}\{(I_{A1} + \dots + O_{C4}), (A_1 + \dots + C_4)\})$ in Fig. 2). This means the theoretical optimization upper limit in the second stage is achieved when all DRAM tensors or computing tiles are continuously executed without stall. BS represents batch size. For the edge platform, the workload is GPT-2-Small, while for the cloud platform, it is GPT-2-XL.

ResNet-50 and GPT-2 are chosen as the default workloads for the discussion due to their representativeness and popularity in image and language processing, respectively.

3) *Baseline*: We select the SOTA Cocco scheduling framework [49] as our baseline, as it explores the layer-fusion space while adopting mainstream tiling and prefetch-and-delayed-storing strategies.

B. Overall Comparisons

Fig. 6 shows that after the first stage, SoMa, on average, improves $1.82\times$ performance and reduces 37.3% energy costs compared to Cocco. The second stage, on average, further improves performance by $1.16\times$ over the first stage. It can be observed that the schemes found in the second stage are very close to the theoretical maximum optimization value (blue diamonds), with an average difference of only 3.1%. Thus, after optimization through the two stages, the best-explored solution improves $2.11\times$ performance and reduces 37.3% energy costs.

SoMa improves performance by $2.15\times$, $2.18\times$, $2.01\times$, $2.61\times$, $2.55\times$ and $1.14\times$ and reduces energy cost by 39.5%, 41.0%, 46.0%, 47.1%, 47.0%, and 3.1% on ResNet-50, ResNet-101, Inception-ResNet-v1, RandWire, GPT2-Prefill, and GPT2-Decode compared to Cocco, respectively. This demonstrates that SoMa can consistently improve performance and energy efficiency on various DNNs. **However, compared to other networks, including GPT-2-Prefill, SoMa demonstrates almost no optimization effect in GPT-2-Decode, and the overall computational utilization remains extremely low.** This is because the decode stage exhibits significantly lower compute density [43], with its latency primarily dominated by weight and KV cache loading. Moreover, another interesting phenomenon is that the computational utilization of GPT-2 does not increase linearly with the batch size; instead, the growth rate gradually diminishes. For instance, after SoMa optimization, the utilization rates

for GPT-2-Small-Decode and GPT-2-XL-Decode across batch sizes of 1, 4, 16, and 64 are 0.66%, 2.03%, 4.26%, 5.84% and 0.60%, 1.90%, 4.13%, 5.83%, respectively. This is because, as the batch size grows, the increasing size of the KV cache becomes comparable to or even exceeds that of the weights, diminishing the benefits of further increases in batch size for improving compute density.

1) *Analysis of the First Stage of SoMa*: In the first stage of SoMa, we observe significant reductions in Core Array Energy and DRAM Energy by 34.8% and 44.3%, respectively. This reduction in Core Array Energy is mainly due to the flexibility of our approach in adjusting the Tiling Number during exploration, unlike Cocco's more conservative approach that selects each tile size based only on the basic parallelism requirements of the computing units [49] (detailed analysis and example can be found in Sec. VII-B1). Therefore, our approach often has fewer tiles within the buffer constraint (with an average total number of computing tiles per network being 7962 for Cocco and 751 for ours), with each tile being larger, allowing more optimization and reuse opportunities in the Core Array Scheduler. The decrease in DRAM Energy is because SoMa allows for a smaller number of LGs per network (an average of 2.5) compared to Cocco (an average of 13.0), meaning it can fuse more layers. The main reasons for this difference are: 1) Cocco's finer-grained tiles result in significant accumulated backtracking halo overlap costs when dealing with convolutional and pooling layers (as introduced in Sec. III-A and Fig. 2). 2) Our approach can use FLCs (with an average of 3.9 FLGs per network) instead of DRAM Cuts to free some weights (as introduced in Sec. IV-A1) and adjust the Tiling Number. Specifically, shuffling weights can save buffer space, enabling the fusion of more layers. Moreover, since different FLGs can have different Tiling Numbers, the FLC can switch the computing granularity between two FLGs without accessing DRAM (detailed analysis in Sec. VII-B1).

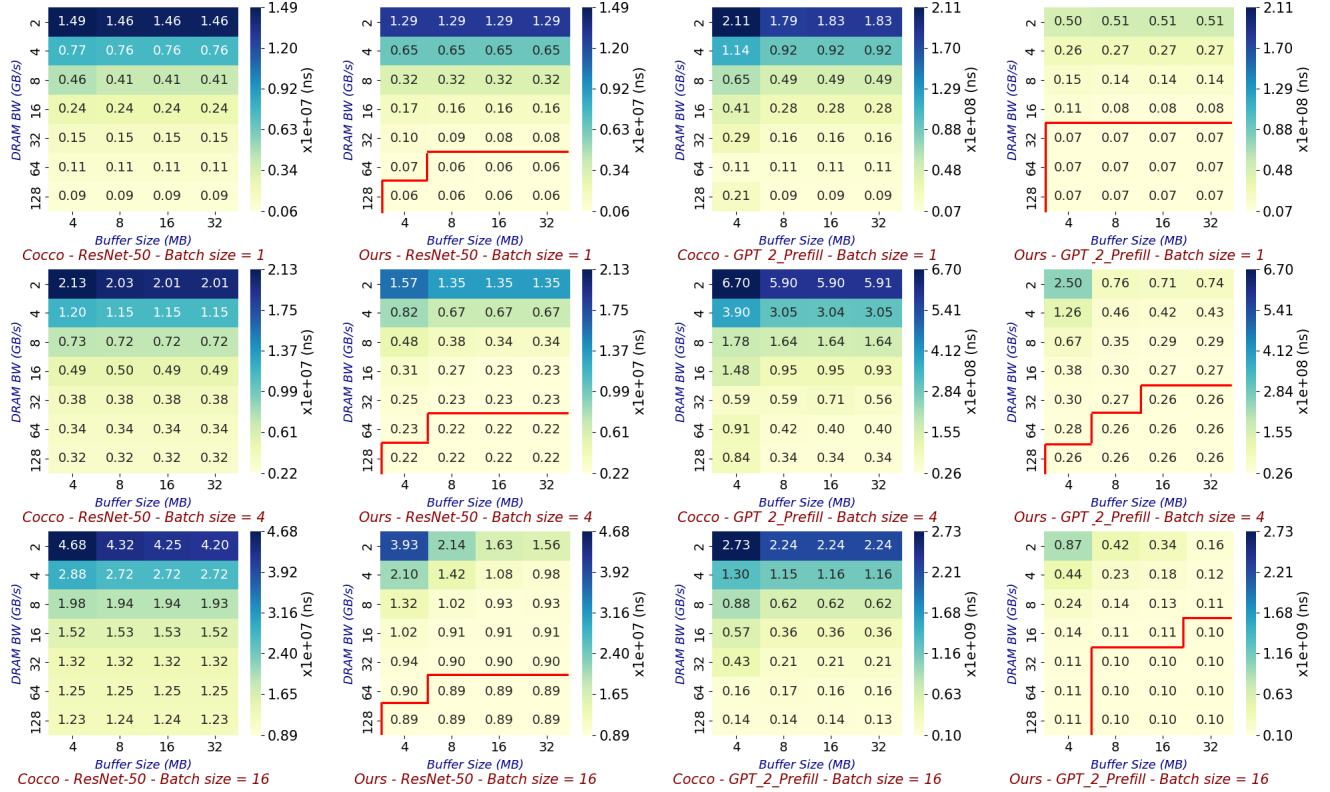


Fig. 7. Design Space Exploration over DRAM Bandwidth and Buffer Size for the 16TOPS Edge DNN Accelerator. We present the latency corresponding to the optimal schemes explored using Cocco and SoMa (Ours) under different networks, batch sizes, DRAM bandwidth, and buffer sizes. In the results of Ours, we highlighted the hardware configurations with the same minimum value (accurate to two decimal places) using a red envelope curve.

The performance improvement also largely stems from the above-analyzed factors, as coarser-grained tiles provide more on-chip reuse opportunities, better overlapping of computation, and GBUF access time, while Cocco may sometimes suffer from GBUF load and store stalls. Additionally, the significant reduction in DRAM accesses naturally reduces overall latency.

2) *Analysis of the Second Stage of SoMa*: The performance improvement brought by the second stage mainly comes from effectively exploiting the DRAM usage imbalance opportunities described in Sec. III-B. Specifically, by intelligently prefetching and delaying storing, many DRAM idle periods can be utilized, reducing computation stalls and DRAM bandwidth waste. In Sec. VII-B2, we demonstrate this optimization through a simple practical example.

VII. DISCUSSION

A. Design Space Exploration

In Fig. 7, we conduct a DSE for a 16 TOPS accelerator across different workloads and batch sizes, focusing on buffer size and DRAM bandwidth. This exploration yields some interesting insights as follows.

1) *Insight 1: When dealing with small batch sizes (especially 1), even with SoMa, DRAM bandwidth plays a more decisive role in performance compared to buffer*

size. However, the impact of buffer size becomes more significant as the batch size increases.

For small batch sizes (e.g., Ours-Batch size = 1 in Fig. 7), the greater influence of DRAM bandwidth is evident as increasing buffer size does not significantly reduce latency, even with SoMa. However, increasing DRAM bandwidth results in a noticeable reduction in latency. This is because: 1) the fmaps sizes are small, so a minimal buffer can almost entirely store data on-chip, making further increases in buffer size unnecessary; 2) weights need to be loaded regardless, but a small batch size means less computation, providing fewer opportunities for SoMa to use prefetching and delayed storing to hide data transfer times. Consequently, weight loading times can severely stall computation and dominate latency. Thus, increasing DRAM bandwidth can directly reduce this latency and significantly enhance overall performance.

As the batch size increases, the impact of buffer size becomes more significant, as evidenced by the greater reduction in latency with larger batch sizes when increasing buffer size (e.g., Ours-Batch size = 4 and 16 in Fig. 7). This is because: 1) the fmaps sizes increase, allowing a larger buffer to better exploit reuse opportunities; 2) additionally, with more computation, the longer computing times provide more opportunities for SoMa to use the buffer for prefetching and delayed storing to hide data transfer times.

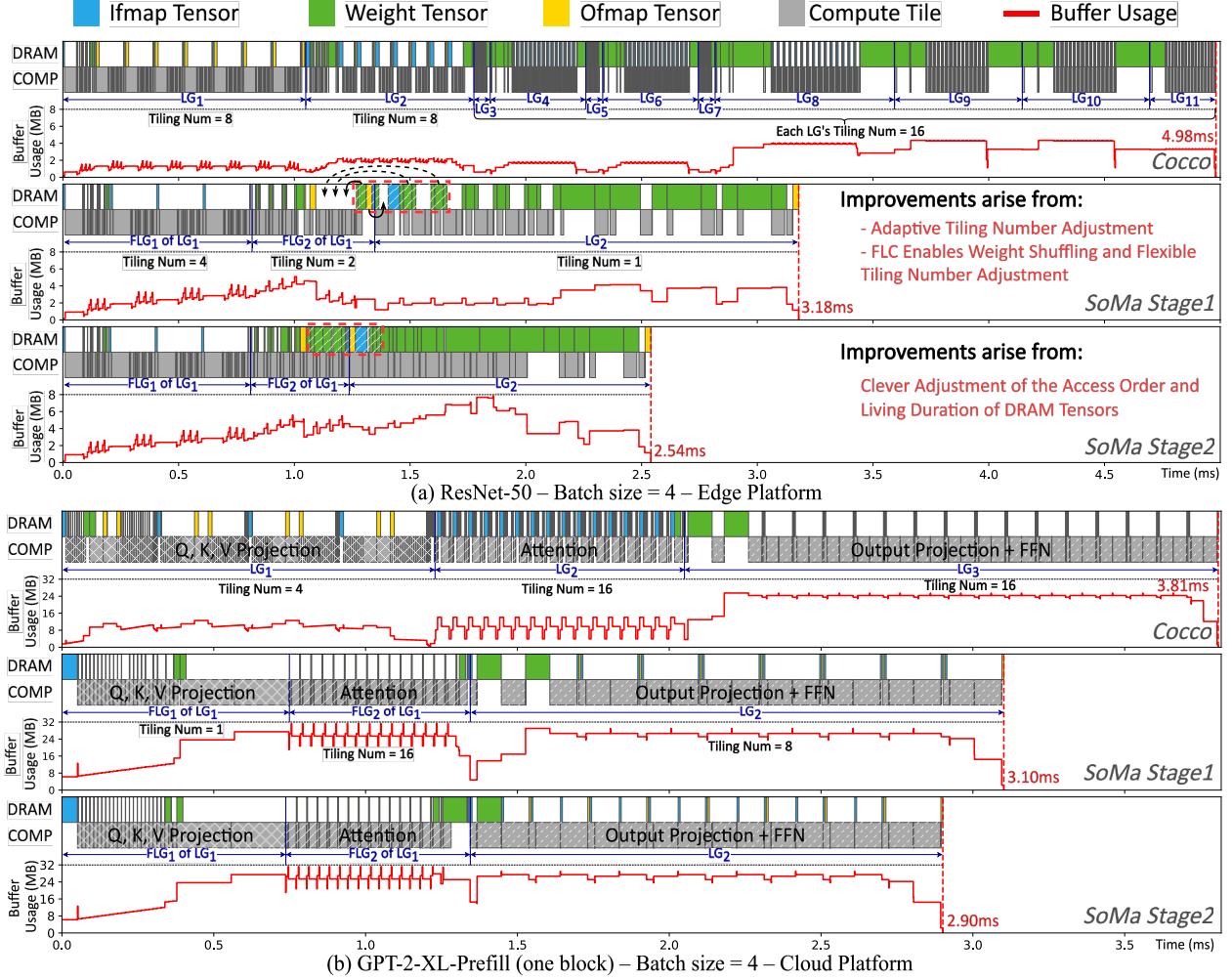


Fig. 8. Comparison of Practical Execution Graphs among the Schemes Explored by Cocco (top), the First Stage (middle), and the Second Stage (bottom) of SoMa on default edge-side accelerator. We also point out all DRAM Cuts, FLCs, and their corresponding Tiling Numbers. In the running graph for one block of GPT-2-XL-Prefill, we have highlighted the main matrix multiplication layers, while certain element-wise layers (such as transpose, softmax, add, and layer normalization) are not explicitly marked. Q, K, and V represents Query, Key, and Value, respectively.

2) **Insight 2: Having both a large buffer and high DRAM bandwidth is often wasteful and offers limited performance gains compared to configurations with either high DRAM bandwidth or moderate DRAM bandwidth paired with a large buffer.**

In Ours in Fig. 7, we observe that many configurations in the lower right corner exhibit similar performance (highlighted by a red envelope curve). Among these points, the ones in the farthest lower right corner are equipped with both the large DRAM bandwidth and buffer sizes, which is quite wasteful. Specifically, we find that for all workloads, there is a noticeable point where increasing DRAM bandwidth beyond a certain threshold yields diminishing returns, as the dominant factor for latency becomes computing time. This lower triangle (red envelope curve) in SoMa, a feature not observed in Cocco, indicates that with the efficient buffer utilization in SoMa, on-chip buffers can adequately compensate for DRAM bandwidth.

While HBM provides high bandwidth density in the current era, its high cost deters many companies [15], [50]. Therefore, moderately increasing buffer size and opting for lower-cost DRAM models could be a viable differentiation strategy, and SoMa can be a significant aid for this.

B. Lessons From a Practical Example

Fig. 8 shows the actual execution graph of scheduling schemes explored by Cocco, SoMa's first stage, and SoMa's second stage, respectively. For ResNet-50, compared to the baseline, the first stage of SoMa achieves an average of $1.57\times$ performance improvement and a 36.1% reduction in energy cost. The second stage achieves an additional average of $1.25\times$ performance improvement over Stage 1, resulting in a total performance gain of $1.96\times$ over Cocco. Next, we will analyze the reasons for SoMa's performance and energy efficiency improvements using this practical example, thereby enhancing

the understanding of the DRAM Communication Scheduling Space.

1) *Lessons From the First Stage:* From this example, we can make two observations: 1) **adaptively adjusting the Tiling Number based on network characteristics and buffer size is crucial for exploiting buffer potential and improving performance;** 2) **FLC can free weights and aggregate fmaps to adjust the Tiling Number without accessing DDR, which can further enhance buffer utilization efficiency and leverage the potential of dynamically adjusting the Tiling Number.** Next, we will analyze the origins of these two insightful observations in detail, using the example in Fig. 8 as a reference.

First, by comparing the Tiling Numbers in Cocco and the first stage of SoMa, we find that Cocco's Tiling Number is larger, while SoMa's Tiling Number can be as low as 1. This is mainly because Cocco uses a heuristic strategy to set the Tiling Number based on the Core Array's parallelism requirements (Kernel-Channel (KC) parallelism [25], [26], [34], [38], [40], [50]), a common approach in many studies [58], [59]. Consequently, larger kernel and channel dimensions result in a higher Tiling Number. For example, in ResNet-50, the Tiling Number (16) for later layers with larger kernel and channel dimensions is higher than that of earlier layers (8). However, this strategy is often conservative, leading to higher Tiling Numbers. A larger Tiling Number increases backtracking halo overlap costs and limits the Core Array Scheduler's ability to find on-chip reuse opportunities, which reduces performance and energy efficiency. In contrast, SoMa better adapts to buffer size and network structure to control the Tiling Number. For example, in ResNet-50, the fmap size decreases as the network deepens, so the buffer consumption for a smaller Tiling Number also decreases accordingly. Thus, SoMa intelligently allows the Tiling Number to decrease monotonically with network depth (from left to right), rather than increasing as in Cocco. Additionally, we observe that in SoMa, the last LG of ResNet-50 and the first FLG of GPT-2-XL-Prefill both have a Tiling Number of 1, allowing for the immediate disposal of weights after processing each tile (or layer). This conserves buffer space and facilitates further fusion.

Second, in both ResNet-50 and GPT-2-XL-Prefill, it happens that the first two LGs in the Cocco scheme and the first two FLGs in the SoMa scheme consist of identical layers. We observe that SoMa efficiently uses the FLC to clear weights and adjust tile sizes without incurring the DRAM access overheads seen in the Cocco scheme. Specifically, in ResNet-50, if the Tiling Number for the first FLG equals 2 (matching the next FLG), the buffer is insufficient for the first FLG. In GPT-2-XL-Prefill, only when the Tiling Number for the first FLG is 1 can it promptly clear weights to accommodate Q, K, and V on chip.

2) *Lessons From the Second Stage:* The improvement of the second stage is attributed to the clever adjustment of the access order and living duration of DRAM tensors. In this stage, SoMa properly performs precise surgical strikes on some key

tensors to reduce the total computing stall.

It can be observed that in both examples, SoMa's second stage effectively prefetches and delays storing DRAM tensors. ResNet-50, however, has more DRAM tensors, making adjustments more complex; therefore, we primarily use it as the example for analysis.

As shown by the black solid arrows near the diagonally striped regions in Fig. 8, DRAM tensors that cause computing stalls are adjusted to DRAM-free periods. More specifically, the weight of the first layer of LG_2 is prefetched several tiles earlier, while the ofmaps of the last layer of FLG_2 is delayed by one tile and swaps positions with the ifmaps of the first layer of LG_2 . This adjustment successfully utilizes DRAM idle time to eliminate the computing stall near the border between FLG_2 and LG_2 , while considering dependencies.

Additionally, it can be observed that LG_2 contains many chunky weights (with the three largest weights up to 2304KB with INT8 precision). Given the limited 8MB buffer, prefetching these weights far in advance is impractical. However, simply pushing all weights forward would not maximally eliminate computing stalls. Therefore, SoMa cleverly selects to move the two relatively larger weight tensors at the front of LG_2 to the DRAM idle time at the back of FLG_2 (marked by dashed black arrows). The remaining weights are not altered in their DRAM Access Order but are pushed forward by changing their Living Duration. This approach maximizes the elimination of computing stalls within LG_2 . Although some computing stalls still appear, the maximum buffer usage near these stalls has already reached 8MB, indicating that further prefetching is not feasible.

VIII. CONCLUSION

In this work, we first analyze various existing paradigms that use buffers to optimize DRAM communication, identifying challenges in the current layer fusion paradigm and uncovering previously overlooked opportunities for prefetching and delayed storing. Based on these observations, we introduce a Tensor-centric Notation and a matching parsing method to describe DRAM communication scheduling schemes, thereby defining the DRAM Communication Scheduling Space. We then propose a two-stage scheduling framework, SoMa, to structurally explore this space. Experimental results demonstrate that, compared to the SOTA Cocco framework, SoMa efficiently explores the broader optimization space defined by our notation, fully utilizing the buffer's potential to optimize DRAM communication. Moreover, we leverage SoMa to conduct several case studies, yielding interesting insights into architecture design and the trade-offs underlying the DRAM Communication Scheduling Space.

IX. ACKNOWLEDGMENT

This research was partially supported by Dushi Program from Tsinghua University.

REFERENCES

- [1] "Soma open-source framework," <https://github.com/SET-Scheduling-Project/SoMa-HPCA2025>.
- [2] D. Abts, G. Kimmell, A. Ling, J. Kim, M. Boyd, A. Bitar, S. Parmar, I. Ahmed, R. DiCecco, D. Han, J. Thompson, M. Bye, J. Hwang, J. Fowers, P. Lillian, A. Murthy, E. Mehtabuddin, C. Tekur, T. Sohmers, K. Kang, S. Maresh, and J. Ross, "A software-defined tensor streaming multiprocessor for large-scale machine learning," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 567–580. [Online]. Available: <https://doi.org/10.1145/3470496.3527405>
- [3] M. Alwani, H. Chen, M. Ferdman, and P. A. Milder, "Fused-layer CNN accelerators," in *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*. IEEE Computer Society, 2016, pp. 22:1–22:12. [Online]. Available: <https://doi.org/10.1109/MICRO.2016.7783725>
- [4] Apple, "Apple a15," https://en.wikipedia.org/wiki/Apple_A15, 2021.
- [5] Apple, "Apple a16," https://en.wikipedia.org/wiki/Apple_A16, 2022.
- [6] P. Bannon, G. Venkataraman, D. D. Sarma, and E. Talpes, "Computer and redundancy solution for the full self-driving computer," in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–22. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875645>
- [7] J. Cai, Y. Wei, Z. Wu, S. Peng, and K. Ma, "Inter-layer scheduling space definition and exploration for tiled accelerators," in *Proceedings of the 50th Annual International Symposium on Computer Architecture, ISCA 2023, Orlando, FL, USA, June 17-21, 2023*, Y. Solihin and M. A. Heinrich, Eds. ACM, 2023, pp. 13:1–13:17. [Online]. Available: <https://doi.org/10.1145/3579371.3589048>
- [8] J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma, "Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 156–171.
- [9] Cambricon, "Mlu290-m5," <https://www.cambricon.com/index.php?m=content&c=index&a=lists&catid=340>.
- [10] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning," in *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 269–284. [Online]. Available: <https://doi.org/10.1145/2541940.2541967>
- [11] Y. Chen, J. S. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 2016, pp. 367–379. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.40>
- [12] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "Dadiannao: A machine-learning supercomputer," in *47th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2014, Cambridge, United Kingdom, December 13-17, 2014*. IEEE Computer Society, 2014, pp. 609–622. [Online]. Available: <https://doi.org/10.1109/MICRO.2014.58>
- [13] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Association for Computational Linguistics, 2019, pp. 4171–4186. [Online]. Available: <https://doi.org/10.18653/v1/n19-1423>
- [14] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021.
- [15] dramexchange, "Dram price," <https://www.dramexchange.com/>.
- [16] A. Firoozshahian, J. Coburn, R. Levenstein, R. Nattoji, A. Kamath, O. Wu, G. Grewal, H. Aepala, B. Jakka, B. Dreyer, A. Hutchin, U. Diril, K. Nair, E. K. Aredestani, M. Schatz, Y. Hao, R. Komuravelli, K. Ho, S. Abu Asal, J. Shajrawi, K. Quinn, N. Sreedhara, P. Kansal, W. Wei, D. Jayaraman, L. Cheng, P. Chopda, E. Wang, A. Bikumandla, A. Karthik Sengottuvel, K. Thottampudi, A. Narasimha, B. Dodds, C. Gao, J. Zhang, M. Al-Sanabani, A. Zehtabioskuie, J. Fix, H. Yu, R. Li, K. Gondkar, J. Montgomery, M. Tsai, S. Dwarakapuram, S. Desai, N. Avidan, P. Ramani, K. Narayanan, A. Mathews, S. Gopal, M. Naumov, V. Rao, K. Noru, H. Reddy, P. Venkatapuram, and A. Bjorlin, "Mtia: First generation silicon targeting meta's recommendation systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: <https://doi.org/10.1145/3579371.3589348>
- [17] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "TETRIS: scalable and efficient neural network acceleration with 3d memory," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2017, Xi'an, China, April 8-12, 2017*, Y. Chen, O. Temam, and J. Carter, Eds. ACM, 2017, pp. 751–764. [Online]. Available: <https://doi.org/10.1145/3037697.3037702>
- [18] L. Gwennap, "Tenstorrent scales ai performance: New multicore architecture leads in data-center power efficiency," 2020. [Online]. Available: <https://www.linleygroup.com/mp/article.php?id=12287>
- [19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016, pp. 770–778. [Online]. Available: <https://doi.org/10.1109/CVPR.2016.90>
- [20] K. Hegde, P. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: enabling efficient algorithm-accelerator mapping space search," in *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Virtual Event, USA, April 19-23, 2021*, T. Sherwood, E. D. Berger, and C. Kozyrakis, Eds. ACM, 2021, pp. 943–958. [Online]. Available: <https://doi.org/10.1145/3445814.3446762>
- [21] Q. Huang, A. Kalaiah, M. Kang, J. Demmel, G. Dinh, J. Wawrzyniak, T. Norell, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 554–566. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00050>
- [22] INTEL, "Goya inference platform white paper," <https://habana.ai/wp-content/uploads/2021/10/Habana-GOYA-Inference-Performance-Whitepaper-Nov20.pdf>.
- [23] M. James, M. Tom, P. Groeneveld, and V. Kibardin, "ISPD 2020 physical mapping of neural networks on a wafer-scale deep learning accelerator," in *ISPD 2020: International Symposium on Physical Design, Taipei, Taiwan, March 29 - April 1, 2020, delayed to September 20-23, 2020*, W. Swartz and J. Lienig, Eds. ACM, 2020, pp. 145–149. [Online]. Available: <https://doi.org/10.1145/3372780.3380846>
- [24] Y. Jiao, L. Han, R. Jin, Y. Su, C. Ho, L. Yin, Y. Li, L. Chen, Z. Chen, L. Liu, Z. He, Y. Yan, J. He, J. Mao, X. Zai, X. Wu, Y. Zhou, M. Gu, G. Zhu, R. Zhong, W. Lee, P. Mao, Y. Chen, W. Li, D. Xiao, Q. Yan, M. Zhuang, J. Chen, Y. Tian, Y. Lin, W. Wu, H. Li, and Z. Dou, "A 12nm programmable convolution-efficient neural-processing-unit chip achieving 825tops," in *2020 IEEE International Solid-State Circuits Conference, ISSCC 2020, San Francisco, CA, USA, February 16-20, 2020*. IEEE, 2020, pp. 136–140. [Online]. Available: <https://doi.org/10.1109/ISSCC19947.2020.9062984>
- [25] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottsch, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. C. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. A. Patterson, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1–14. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00010>
- [26] N. P. Jouppi, C. Young, N. Patil, D. A. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani,

- C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture, ISCA 2017, Toronto, ON, Canada, June 24-28, 2017*. ACM, 2017, pp. 1–12. [Online]. Available: <https://doi.org/10.1145/3079856.3080246>
- [27] S. Kao, G. Jeong, and T. Krishna, "Confucius: Autonomous hardware resource assignment for DNN accelerators using reinforcement learning," in *53rd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2020, Athens, Greece, October 17-21, 2020*. IEEE, 2020, pp. 622–636. [Online]. Available: <https://doi.org/10.1109/MICRO50266.2020.00058>
- [28] S. Kao and T. Krishna, "GAMMA: automating the HW mapping of DNN models on accelerators via genetic algorithm," in *IEEE/ACM International Conference On Computer Aided Design, ICCAD 2020, San Diego, CA, USA, November 2-5, 2020*. IEEE, 2020, pp. 44:1–44:9. [Online]. Available: <https://doi.org/10.1145/3400302.3415639>
- [29] S.-C. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "Flat: An optimized dataflow for mitigating attention bottlenecks," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, 2023*, pp. 295–310.
- [30] D. Kim, J. Kung, S. M. Chai, S. Yalamanchili, and S. Mukhopadhyay, "Neurocube: A programmable digital neuromorphic architecture with high-density 3d memory," in *43rd ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2016, Seoul, South Korea, June 18-22, 2016*. IEEE Computer Society, 2016, pp. 380–392. [Online]. Available: <https://doi.org/10.1109/ISCA.2016.41>
- [31] S. Knowles, "Graphcore," in *IEEE Hot Chips 33 Symposium, HCS 2021, Palo Alto, CA, USA, August 22-24, 2021*. IEEE, 2021, pp. 1–25. [Online]. Available: <https://doi.org/10.1109/HCS52781.2021.9567075>
- [32] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 2019, pp. 754–768. [Online]. Available: <https://doi.org/10.1145/3352460.3358252>
- [33] J. Lee, D. Shin, J. Lee, J. Lee, S. Kang, and H. Yoo, "A full HD 60 fps CNN super resolution processor with selective caching based layer fusion for mobile devices," in *2019 Symposium on VLSI Circuits, Kyoto, Japan, June 9-14, 2019*. IEEE, 2019, p. 302. [Online]. Available: <https://doi.org/10.23919/VLSIC.2019.8778104>
- [34] H. Liao, J. Tu, J. Xia, and X. Zhou, "Davinci: A scalable architecture for neural network computing," in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–44. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875654>
- [35] D. Liu, T. Chen, S. Liu, J. Zhou, S. Zhou, O. Teman, X. Feng, X. Zhou, and Y. Chen, "Pudiannao: A polyvalent machine learning accelerator," *SIGPLAN Not.*, vol. 50, no. 4, p. 369–381, mar 2015. [Online]. Available: <https://doi.org/10.1145/2775054.2694358>
- [36] L. Lu, N. Guan, Y. Wang, L. Jia, Z. Luo, J. Yin, J. Cong, and Y. Liang, "TENET: A framework for modeling tensor dataflow based on relation-centric notation," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 720–733. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00062>
- [37] L. Mei, K. Goetschalckx, A. Symons, and M. Verhelst, "Defines: Enabling fast exploration of the depth-first scheduling space for dnn accelerators through analytical modeling," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 570–583.
- [38] T. Norrie, N. Patil, D. H. Yoon, G. Kurian, S. Li, J. Laudon, C. Young, N. P. Jouppi, and D. A. Patterson, "Google's training chips revealed: Tpuv2 and tpuv3," in *IEEE Hot Chips 32 Symposium, HCS 2020, Palo Alto, CA, USA, August 16-18, 2020*. IEEE, 2020, pp. 1–70. [Online]. Available: <https://doi.org/10.1109/HCS49909.2020.9220735>
- [39] Nvidia, "Hardware for self-driving cars," <https://www.nvidia.com/en-us/self-driving-cars/drive-platform/hardware/>.
- [40] Nvidia, "Nvidia deep learning accelerator," <http://nvidia.org>, 2017.
- [41] OpenAI, "Gpt-4 technical report," 2023.
- [42] A. Parashar, P. Raina, Y. S. Shao, Y. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. S. Emer, "Timeloop: A systematic approach to DNN accelerator evaluation," in *IEEE International Symposium on Performance Analysis of Systems and Software, ISPASS 2019, Madison, WI, USA, March 24-26, 2019*. IEEE, 2019, pp. 304–315. [Online]. Available: <https://doi.org/10.1109/ISPASS.2019.00042>
- [43] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Gouri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [44] Qualcomm, "Qualcomm snapdragon 8gen3," [https://en.wikipedia.org/wiki/List_of_Qualcomm_Snapdragon_systems_on_chips#Snapdragon_8/8s_Gen_3_\(2024\)](https://en.wikipedia.org/wiki/List_of_Qualcomm_Snapdragon_systems_on_chips#Snapdragon_8/8s_Gen_3_(2024)), 2024.
- [45] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [46] Z. Sun, S. Kvatinsky, X. Si, A. Mehonic, Y. Cai, and R. Huang, "A full spectrum of computing-in-memory technologies," *Nature Electronics*, pp. 1–13, 2023.
- [47] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-resnet and the impact of residual connections on learning," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, February 4-9, 2017, San Francisco, California, USA, S. P. Singh and S. Markovitch, Eds.* AAAI Press, 2017, pp. 4278–4284. [Online]. Available: <http://aaai.org/ocs/index.php/AAAI/AAAI17/paper/view/14806>
- [48] Z. Tan, H. Cai, R. Dong, and K. Ma, "Nn-baton: DNN workload orchestration and chiplet granularity exploration for multichip accelerators," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1013–1026. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00083>
- [49] Z. Tan, Z. Zhu, and K. Ma, "Cocco: Hardware-mapping co-exploration towards memory capacity-communication optimization," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ser. ASPLOS '24*. New York, NY, USA: Association for Computing Machinery, 2024, p. 69–84. [Online]. Available: <https://doi.org/10.1145/3617232.3624865>
- [50] J. Vasiljevic, L. Bajic, D. Capalija, S. Sokorac, D. Ignjatovic, L. Bajic, M. Trajkovic, I. Hamer, I. Matosevic, A. Cejkov, U. Aydonat, T. Zhou, S. Z. Gilani, A. Paiva, J. Chu, D. Maksimovic, S. A. Chin, Z. Moudallal, A. Rakhmat, S. Nijjar, A. Bhullar, B. Drazic, C. Lee, J. Sun, K. Kwong, J. Connolly, M. Dooley, H. Farooq, J. Y. T. Chen, M. Walker, K. Dabiri, K. Mabee, R. S. Lal, N. Rajatheva, R. Retnamma, S. Karodi, D. Rosen, E. Munoz, A. Lewycky, A. Knezevic, R. Kim, A. Rui, A. Drouillard, and D. Thompson, "Compute substrate for software 2.0," *IEEE Micro*, vol. 41, no. 2, pp. 50–55, 2021. [Online]. Available: <https://doi.org/10.1109/MM.2021.3061912>
- [51] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems, ser. NIPS'17*. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.
- [52] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. R. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. S. Emer, S. W. Keckler, and B. Khailany, "Magnet: A modular accelerator generator for neural networks," in *Proceedings of the International Conference on Computer-Aided Design, ICCAD 2019, Westminster, CO, USA, November 4-7, 2019, D. Z. Pan, Ed.* ACM, 2019, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/ICCAD45719.2019.8942127>
- [53] O. Wechsler, M. Behar, and B. Daga, "Spring hill (NNP-I 1000) intel's data center inference chip," in *2019 IEEE Hot Chips 31 Symposium (HCS), Cupertino, CA, USA, August 18-20, 2019*. IEEE, 2019, pp. 1–12. [Online]. Available: <https://doi.org/10.1109/HOTCHIPS.2019.8875671>
- [54] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, pp. 20–24, 1995. [Online]. Available: <https://doi.org/10.1145/216585.216588>
- [55] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang, "HASCO: towards agile hardware and software co-design for tensor computation," in *48th ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2021, Valencia, Spain, June 14-18, 2021*. IEEE, 2021, pp. 1055–1068. [Online]. Available: <https://doi.org/10.1109/ISCA52012.2021.00086>

- [56] S. Xie, A. Kirillov, R. Girshick, and K. He, “Exploring randomly wired neural networks for image recognition,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1284–1293.
- [57] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz, “Interstellar: Using halide’s scheduling language to analyze DNN accelerators,” in *ASPLOS ’20: Architectural Support for Programming Languages and Operating Systems, Lausanne, Switzerland, March 16-20, 2020*, J. R. Larus, L. Ceze, and K. Strauss, Eds. ACM, 2020, pp. 369–383. [Online]. Available: <https://doi.org/10.1145/3373376.3378514>
- [58] S. Zheng, X. Zhang, L. Liu, S. Wei, and S. Yin, “Atomic dataflow based graph-level workload orchestration for scalable DNN accelerators,” in *IEEE International Symposium on High-Performance Computer Architecture, HPCA 2022, Seoul, South Korea, April 2-6, 2022*. IEEE, 2022, pp. 475–489. [Online]. Available: <https://doi.org/10.1109/HPCA53966.2022.00042>
- [59] S. Zheng, X. Zhang, D. Ou, S. Tang, L. Liu, S. Wei, and S. Yin, “Efficient scheduling of irregular network structures on cnn accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3408–3419, 2020.
- [60] S. Zheng, S. Chen, P. Song, R. Chen, X. Li, S. Yan, D. Lin, J. Leng, and Y. Liang, “Chimera: An analytical optimizing framework for effective compute-intensive operators fusion,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1113–1126.

APPENDIX

A. Abstract

This appendix provides guidance on accessing and using the SoMa framework (introduced in Sec. V) to replicate the key results shown in Fig. 6 and Fig. 7. It also presents the architecture of the end-to-end compiler built on SoMa, detailing the inputs and outputs of each major stage (from model input to final instructions). Other experiments, which also involve similar steps and analyses, are omitted here for the sake of brevity.

B. Artifact check-list (meta-information)

- **Algorithm:** Simulated Annealing
- **Program:** C++, Shell, Python (only for data collection)
- **Compilation:** by Makefile
- **Hardware:** Recommend a server with 96+ cores and at least 1GB RAM per core.
- **Metrics:** Cost function $E \times D$ is employed in all experiments.
- **Experiments:** reproduce Fig. 7 and Fig. 6.
- **How much disk space required (approximately)?:** 1GB
- **How much time is needed to prepare workflow(approximately)?:** Several minutes at most.
- **How much time is needed to complete experiments (approximately)?:** For all 432 experiments (96 for Fig. 6 and 332 for Fig. 7), it takes about 2 days on a 192-core Intel Xeon Platinum 8260. Most experiments (95%) are completed within 3.5 hours, while the remaining ones, mainly experiments with batch=64, require the full 2 days to finish.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** AGPL-3.0 License
- **Archived (provide DOI)?:** 10.5281/zenodo.14599935

C. Description

1) *How to access:* The artifact is uploaded to Zenodo:10.5281/zenodo.14599935

2) *Software dependencies:* A C++ compilation environment with support for the C++ 17 standard is required. Linux is recommended. It is recommended to use “GNU make” to build the program. Additionally, the following Python packages are required for reproducing Fig. 7: “pandas”, “matplotlib”, “seaborn”, and “numpy”.

D. Installation

For artifact evaluation, start by downloading the artifact from Zenodo:

```
$ wget -O SOMA_AE.zip https://zenodo.org/records/14599935/files/SOMA_AE.zip?download=1
$ unzip SOMA_AE.zip
```

Our SoMa exploration framework is in “SOMA”. We use ‘build.sh’ to build the SoMa framework and create the result directory.

```
$ cd SOMA
$ ./build.sh
```

The executable target will be generated at “./build/soma”, and the result directories will be “results/overall” and “results/dse”.

You can install the needed Python packages using pip with the following commands.

```
$ pip install -r requirements.txt
```

Or you can use conda to install with the following commands.

```
$ conda install --file requirements.txt
```

E. Experiment workflow

1) *Overall and DSE:* Once the SoMa framework is built, you can reproduce the Overall (Fig. 6) and DSE (Fig. 7) experiments with the command below.

```
$ ./run.sh --eta
```

The “run.sh” takes parameters from “args.txt” as input for each soma instance, including compute power, DRAM bandwidth, storage directory, and the random seed. For each configuration, both our method and the baseline use the same seed. By default, the “run.sh” utilizes all CPU cores, with each core running a separate SoMa process. Each SoMa process outputs the corresponding results and logs to “results/overall” and “results/dse”. When using all 192 cores, it takes around 2 days to run on an Intel Xeon Platinum 8260 server. Due to the long runtime, we recommend using tools like “nohup” or “screen” to prevent disconnection due to inactivity, ensuring that all experiments complete successfully.

After all experiments are completed, we use “get_results.sh” to extract the results from the raw outputs.

```
$ ./get_results.sh
```

“get_results.sh” will use Python scripts under folder “pyscripts” to generate four files: “overall.csv”, “stats.log”, “dse.csv”, and “Fig7_heatmaps_DSE.svg”. “overall.csv” contains all the data presented in Fig. 6, while “stats.log” includes all the data analyzed and calculated in the Sec. VI-B. “dse.csv” contains all the data related to Fig. 7, and “Fig7_heatmaps_DSE.svg” reproduces Fig. 7.

2) *Comparison with Baselines*: The comparison with the baseline in Fig. 6 can be found in “stats.log”. For detailed data of each case, you can refer to “overall.csv”. The results of the DSE experiment are available in “Fig7_heatmaps_DSE.svg”, with the detailed data in “dse.csv”, which is also human-readable, just like “overall.csv”.

3) *SoMa Compiler Workflow*: In “Compiler-IR”, we present open-sourced files that showcase the workflow of the end-to-end SoMa-based compiler developed for our high-performance commercial AI accelerator, the ZEBU FPGA-based Verification Platform, and the corresponding results. For related information, please refer to “Compiler-IR/Readme.md”. (This section is for material demonstration only and does not require execution or reproduction.)

While we are currently unable to release the full source code of the whole compiler due to IP flow restrictions, we believe the provided scheduling engine at the core of SoMa, along with the materials and documentation, effectively offers a clear understanding of the entire workflow of the SoMa-based compiler. Additionally, we are committed to establishing a small-scale cloud platform after the chip tape-out and related testing are completed. This platform will allow users to access the open-sourced compiler based on SoMa, with the flexibility to modify or even replace our scheduler (as long as the output is converted into IR format), enabling translation into chip-executable instructions. (This commitment has also been included in Sec. V-F of the paper.)

F. Evaluation and expected results

After executing “./get_results.sh”, you can use the following script to compare the results with the expected ones:

```
$ compare.sh
```

“compare.sh” uses the “diff” command to compare “overall.csv”, “stats.log”, and “dse.csv”. The “Fig7_heatmaps_DSE.svg” file may not be byte-for-byte identical due to font or library version differences, but the data used, namely “dse.csv”, should be identical. If all files match, it will output a message like “All files match the expected results!”. Otherwise, it will report “Some files do not match the expected results.” and highlight the differences.

G. Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-badging>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>